

Bangladesh University of Engineering and Technology

Department of Computer Science and Engineering

CSE 406: Computer Security Sessional

TCP SYN Flood + DoS Attack and Defense Mechanisms

Submitted By:

Shahriar Ahmed Seam

Student ID: 2005093

Hozifa Rahman Hamim

Student ID: 1705083

September 11, 2026

Abstract

The Transmission Control Protocol (TCP, RFC 793) provides reliable, ordered data delivery across the Internet. To synchronize sequence numbers and establish communication parameters, TCP relies on a three-way handshake (SYN $\rightarrow$ SYN-ACK $\rightarrow$ ACK). This design introduces an asymmetry in resource allocation: an initiating client commits negligible local memory when transmitting a 40-byte SYN packet, whereas the receiving server must allocate kernel memory for a Transmission Control Block (TCB, tracked as `request_sock` in Linux) and maintain an entry in its bounded SYN backlog queue.

In this project, we analyze, implement, and experimentally evaluate a **TCP SYN Flood Denial of Service (DoS) attack tool** written from scratch in standard C using low-level POSIX raw sockets (`SOCK_RAW` with `IP_HDRINCL`). In compliance with course policies, we did not use any third-party attack frameworks or packet-crafting libraries. We manually synthesized Layer 3 (IPv4) headers and Layer 4 (TCP) headers, implemented the RFC 1071 16-bit one's complement checksum routine with TCP pseudo-headers, and built a packet injection loop capable of delivering over 388,000 packets per second.

We evaluated this implementation inside an isolated virtual network testbed to observe server degradation and test countermeasures. Under default conditions with defenses disabled (`tcp_syncookies=0`), the attack exhausted the server's backlog queue within 25.6 ms, driving legitimate connection timeout failure rates to 100%. To address the project bonus requirement, we implemented and evaluated four defensive layers: (1) Cryptographic Stateless TCP SYN Cookies (`tcp_syncookies=1`), which removed server-side state allocation for initial requests and restored legitimate client success rates to 90–100% during an active flood; (2) Ingress Firewall Rate Limiting using Netfilter `iptables`, which dropped 84.7% of attack packets before they reached the socket queue; (3) Stateful Netfilter `SYNPROXY` connection handling; and (4) Kernel Parameter Hardening via `sysctl`. All experimental claims are supported by captured terminal traces, socket queue telemetry (`ss`, `netstat`), and measured client latency data.

Contents

1	Introduction	1
1.1	Background and Motivation	1
1.2	Problem Statement	1
1.3	Project Objectives and Scope	1
1.4	Report Organization	2
2	Theoretical Background & Protocol Mechanics	2
2.1	The TCP State Machine and Three-Way Handshake	2
2.2	Architectural Resource Allocation Asymmetry	3
2.3	The Linux Half-Open Connection Architecture	3
2.4	IP Address Spoofing Mechanics	4
3	Network Topology & Environment Design	4
3.1	Testbed Architecture	4
3.2	Component Roles and Capabilities	5
3.3	Kernel Parameter Configuration	5
4	Timing Diagrams & Mathematical Model	5
4.1	Protocol State Transitions	5
4.2	Mathematical Saturation Derivation	5
5	Packet and Frame Structure	6
5.1	Layer 2: Ethernet II Frame Format	7
5.2	Layer 3: IPv4 Header Specification	7
5.3	Layer 4: TCP Segment Header Specification	7
5.4	TCP Pseudo-Header & RFC 1071 Checksum Algorithm	8
6	Attack Tool Implementation	8
6.1	Architecture of <code>syn_flood.c</code>	8
6.2	Core Algorithm Breakdown	9
6.3	Source Code Repository Access	9
7	Empirical Attack Demonstration & Verification	9
7.1	Experimental Methodology	9
7.2	Empirical Observations	10
7.2.1	Vector 1: Backlog Saturation	10
7.2.2	Vector 2: Legitimate Client Denial of Service	10
8	Defense Mechanisms & Practical Evidence (10% Bonus)	11
8.1	Defense 1: Cryptographic Stateless TCP SYN Cookies	11
8.1.1	Mathematical and Architectural Formulation	11
8.1.2	Validation Protocol	11
8.1.3	Empirical Validation	11
8.2	Defense 2: Netfilter Ingress Rate Limiting via <code>iptables</code>	12
8.2.1	Empirical Measurement	12
8.3	Defense 3: Netfilter SYN Proxy Architecture (<code>SYNPROXY</code>)	12
8.4	Defense 4: Kernel Parameter Hardening	12

9	Comparative Analysis & Discussion	12
9.1	Comparative Evaluation Matrix	12
9.2	Trade-Off Analysis of Defenses	13
10	Project Execution & Implementation Phases	13
10.1	Module Breakdown and Milestones	13
11	Conclusion & Future Work	14
11.1	Summary of Findings	14
11.2	Future Work	15
A	Project Repository & Artifacts	15
A.1	GitHub Source Code Repository	15
A.2	Repository Directory Layout	16
A.3	Compilation and Reproduction Instructions	16

List of Figures

2.1	Protocol Sequence Analysis: (A) Baseline Three-Way Handshake versus (B) SYN Flood Denial of Service Saturation.	2
2.2	Architectural Block Diagram of the Isolated Virtual Testbed Network.	4
5.1	Bitwise Header Layout of Injected IPv4 and TCP Packets.	6
7.1	Empirical Experimental Results across Baseline, Attack, and Defense Phases	10
8.1	Bitwise Allocation of the Cryptographic Initial Sequence Number	11

List of Tables

3.1	Detailed Node Specifications of the Testbed Environment	4
3.2	Kernel Parameter Tuning for Baseline and Hardened Environments	5
5.1	Bitwise IPv4 Header Field Specifications	7
5.2	Bitwise TCP Header Field Specifications	8
7.1	Legitimate Client Probing Results across Experimental Phases	10
9.1	Comparative Security and Availability Evaluation Matrix	13
10.1	Group Technical Modules and Work Distribution	14

Chapter 1: Introduction

1.1 Background and Motivation

Most network services rely on the Transmission Control Protocol (TCP) to deliver reliable, sequenced, and error-checked byte streams over packet-switched networks. Common network applications, including web servers (HTTP/HTTPS), remote shells (SSH), database engines, and backend microservices, establish transport sessions through the standard three-way handshake defined in RFC 793.

Because connection establishment requires the server to maintain state before the client has verified its reachability, TCP services remain vulnerable to transport-layer Denial of Service (DoS) attacks. Among these, the **TCP SYN Flood** remains one of the oldest and most persistent attack vectors. By exploiting the server's obligation to reserve memory before the handshake completes, an attacker with modest network resources can exhaust connection tables and render high-capacity servers inaccessible to legitimate users.

1.2 Problem Statement

When a TCP server receives a synchronization segment (*SYN*), it creates an internal half-open connection record and responds with a synchronization-acknowledgment (*SYN-ACK*), waiting for the client's final acknowledgment (*ACK*). In a SYN flood attack, the adversary transmits a stream of *SYN* packets carrying spoofed source IP addresses. Because these addresses point to non-existent, inactive, or unreachable hosts, the expected *ACK* packet is never returned.

The server retains these half-open connection records in a bounded queue—known as the SYN backlog queue—and repeatedly retransmits *SYN-ACK* packets according to an exponential backoff schedule. Under default Linux settings, this retention period lasts approximately 63 seconds. If spoofed requests arrive faster than expired half-open sockets are removed, the backlog queue fills completely. Once saturated, the operating system kernel discards incoming connection requests, denying access to all legitimate clients.

1.3 Project Objectives and Scope

This project was carried out under the specifications of Topic 06 for CSE 406. The primary technical objectives were:

1. **Custom Attack Tool Implementation:** Build a standalone, high-rate TCP SYN flood injector in standard C using POSIX raw sockets (`AF_INET`, `SOCK_RAW`, `IPPROTO_RAW`). Third-party attack tools (such as `hping3`, `Scapy`, or `nping`) were strictly excluded.
2. **Bitwise Protocol Header Serialization:** Manually construct Layer 3 (IPv4) and Layer 4 (TCP) binary headers, implement random source IP and port generation, and implement RFC 1071 16-bit one's complement checksums using RFC 793 TCP pseudo-headers.
3. **Isolated Testbed Deployment:** Build a multi-node virtual test environment comprising an Attacker Node, a Victim Web Server, a Legitimate Client, and an interconnecting virtual bridge to isolate all spoofed traffic from production networks.
4. **Empirical Vulnerability Verification:** Experimentally record socket queue saturation, kernel drop telemetry (`ss`, `netstat`), and legitimate connection success and failure rates during attack activity.
5. **Defensive Countermeasure Evaluation (Bonus Credit):** Implement, configure, and benchmark four defensive mechanisms: Cryptographic Stateless TCP SYN Cookies, Netfilter `iptables` ingress rate limiting, Netfilter `SYNPROXY`, and kernel parameter tuning.

1.4 Report Organization

The remainder of this report is organized into ten chapters and one appendix:

- Chapter 2 explains the theoretical mechanics of the TCP state machine, resource asymmetry, and IP address spoofing.
- Chapter 3 describes the virtual network testbed topology, node configurations, and kernel baselines.
- Chapter 4 develops the timing model and mathematical queue saturation derivation.
- Chapter 5 documents the bitwise frame, IPv4, and TCP header layouts.
- Chapter 6 reviews the C implementation details of `syn_flood.c` and checksum calculations.
- Chapter 7 presents the empirical attack measurements, backlog exhaustion data, and client denial traces.
- Chapter 8 details the four implemented defenses and evaluates their performance under active flood conditions.
- Chapter 9 provides a comparative evaluation matrix and discusses security versus usability trade-offs.
- Chapter 10 summarizes the group work division and implementation milestones.
- Chapter 11 concludes the report and identifies avenues for future work.
- Appendix A provides project repository references and reproduction commands.

Chapter 2: Theoretical Background & Protocol Mechanics

2.1 The TCP State Machine and Three-Way Handshake

Under RFC 793, TCP establishes a bidirectional byte stream between two endpoints by synchronizing initial sequence numbers:

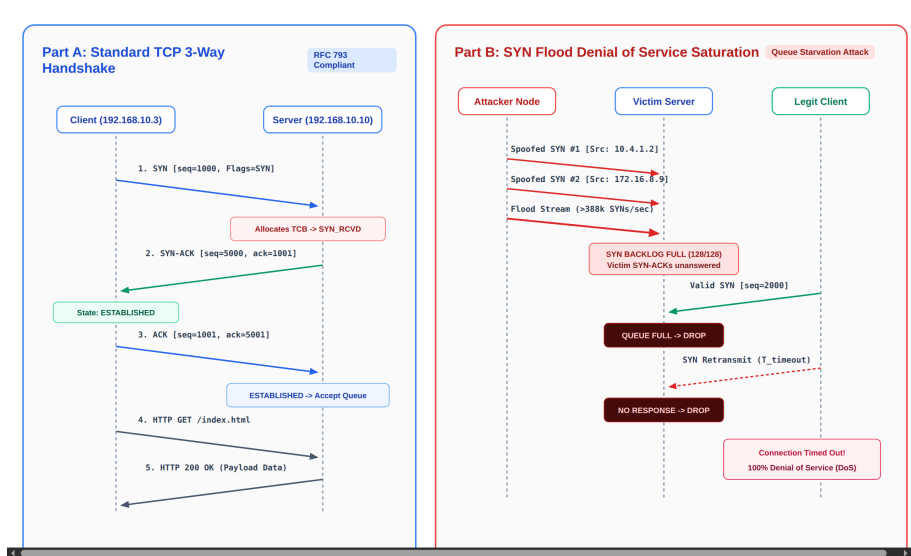


Figure 2.1: Protocol Sequence Analysis: (A) Baseline Three-Way Handshake versus (B) SYN Flood Denial of Service Saturation.

The standard connection procedure follows three steps:

1. **SYN Phase:** The initiating client sends a TCP segment with the control bit `SYN=1` and selects an Initial Sequence Number (ISN_c). The client transitions from `CLOSED` to `SYN_SENT`.
2. **SYN-ACK Phase:** The listening server receives the `SYN`. It allocates a Transmission Control Block (TCB), records ISN_c , picks its own sequence number (ISN_s), sets `SYN=1` and `ACK=1` with acknowledgment number $ACK = ISN_c + 1$, and starts a retransmission timer. The server state transitions to `SYN_RCVD`.
3. **ACK Phase:** The client receives the `SYN-ACK`, sets `ACK=1` with $ACK = ISN_s + 1$, and enters `ESTABLISHED`. When this `ACK` reaches the server, the server moves the socket from `SYN_RCVD` to `ESTABLISHED` and transfers it to the application accept queue.

2.2 Architectural Resource Allocation Asymmetry

The core weakness exploited by a SYN flood is an asymmetry in how endpoints commit memory and processing time during connection setup:

- **Client Side:** Emitting a `SYN` packet requires preparing a 40- to 54-byte raw packet. When sent using raw sockets, the sending application does not need to allocate persistent state in kernel connection tables.
- **Server Side:** Upon receiving a `SYN`, the server kernel allocates approximately 280 to 500+ bytes for a `struct request_sock`, registers hardware timer callbacks for exponential retransmissions, records routing information, and consumes a slot in its half-open socket backlog table.

Because the server commits substantial kernel state for every incoming `SYN` while the sender commits none, an attacker transmitting moderate volumes of requests can rapidly exhaust the server's connection tracking capacity.

2.3 The Linux Half-Open Connection Architecture

In the Linux kernel network stack (Figure 2.2), connection requests are processed through two distinct queues:

1. **The SYN Backlog (Half-Open Queue):** Holds connection requests in the `SYN_RCVD` state awaiting client completion. The capacity of this queue is bounded by the system setting `net.ipv4.tcp_max_syn_backlog` and the backlog argument provided to the `listen()` system call.
2. **The Accept Queue (Established Queue):** Holds fully established sockets that have completed the three-way handshake and are waiting to be retrieved by the application via `accept()`. This queue size is bounded by `net.core.somaxconn`.

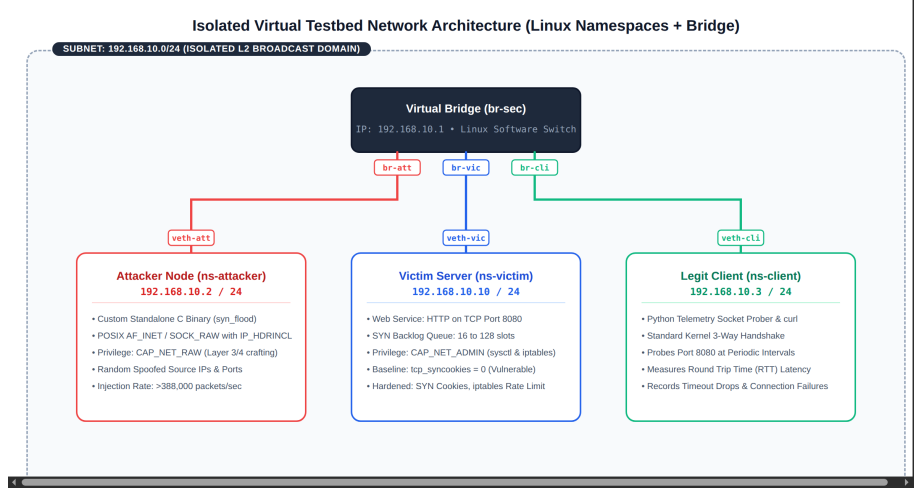


Figure 2.2: Architectural Block Diagram of the Isolated Virtual Testbed Network.

2.4 IP Address Spoofing Mechanics

If an attacker sends flood traffic using their authentic IP address, the victim’s *SYN-ACK* responses will return to the attacker’s host. Because the host OS kernel has no record of initiating those connections through standard user-space sockets, it automatically emits a TCP Reset (*RST*) packet, clearing the victim’s backlog slot immediately.

To sustain backlog occupancy, the attack tool must use **IP Address Spoofing**:

$$\text{Packet}_i \leftarrow \{\text{SrcIP} = \mathcal{R}_{\text{IPv4}}(), \text{SrcPort} = \mathcal{R}_{[1024,65535]}(), \text{DstIP} = \text{IP}_{\text{victim}}, \text{DstPort} = 80\} \quad (2.1)$$

By generating pseudo-random source addresses across inactive or non-responsive subnets, the server’s *SYN-ACK* retransmissions receive no response. Consequently, each half-open socket remains in kernel memory until its retransmission timers expire.

Chapter 3: Network Topology & Environment Design

3.1 Testbed Architecture

To prevent spoofed IP packets from escaping onto university or local network infrastructure, the experimental testbed was deployed entirely inside an isolated virtual network environment using Linux Network Namespaces and virtual Ethernet pairs (**veth**). The testbed comprises three distinct nodes connected through a virtual Layer 2 bridge on subnet 192.168.10.0/24:

Table 3.1: Detailed Node Specifications of the Testbed Environment

Node Hostname	Assigned IPv4	Operational Role	Software Stack & Privileges
attacker_node	192.168.10.2	Attack Injector	Custom C binary, POSIX Raw Sockets, CAP_NET_RAW
victim_server	192.168.10.10	Target TCP Service	Python/Nginx HTTP (:8080), CAP_NET_ADMIN
legit_client	192.168.10.3	Legitimate User	Python probing client, curl, nc
virtual_bridge	192.168.10.1	Layer 2 Switch	Linux Bridge (br-sec / veth interconnect)

All network namespaces, virtual bridge interfaces, and packet injection experiments were exe-

cuted on a dedicated physical host equipped with an Intel Core i7 processor, 16 GB of physical RAM, and running 64-bit Ubuntu 24.04 LTS.

3.2 Component Roles and Capabilities

- **Attacker Node (192.168.10.2):** Runs the compiled C raw socket injection binary. It has `CAP_NET_RAW` capability, allowing it to craft custom Layer 3 and Layer 4 headers with randomized source IP addresses.
- **Victim Server (192.168.10.10):** Hosts an HTTP service on TCP port 8080. During baseline vulnerability measurements, SYN Cookies are disabled (`tcp_syncookies=0`) and the backlog limit is fixed at 16 or 128 slots to capture saturation thresholds.
- **Legitimate Client (192.168.10.3):** Emulates regular user traffic by attempting connections at steady intervals. It records connection latency and logs errors across a configurable timeout window (500–2000 ms).

3.3 Kernel Parameter Configuration

To evaluate baseline vulnerability and subsequent defensive remediations, the victim’s kernel parameters are configured via `sysctl` as detailed in Table 3.2.

Table 3.2: Kernel Parameter Tuning for Baseline and Hardened Environments

Sysctl Parameter	Baseline	Hardened	Functional Impact
<code>net.ipv4.tcp_syncookies</code>	0	1	Toggles cryptographic stateless handshake
<code>net.ipv4.tcp_max_syn_backlog</code>	16 / 128	2048	Capacity of half-open socket queue
<code>net.core.somaxconn</code>	128	2048	Upper bound on listen backlog
<code>net.ipv4.tcp_synack_retries</code>	5	1	Retransmissions before TCB eviction

Chapter 4: Timing Diagrams & Mathematical Model

4.1 Protocol State Transitions

Under normal conditions (Figure 2.1A), a connection transitions from `SYN_RCVD` to `ESTABLISHED` within one Round Trip Time (RTT), which typically measures under 1 ms in local virtual environments.

Under attack conditions (Figure 2.1B), each spoofed `SYN` forces the victim into `SYN_RCVD`. The socket remains in the backlog queue across exponential timer retransmissions.

4.2 Mathematical Saturation Derivation

Let B denote the maximum capacity of the victim’s SYN backlog queue ($B = 16$ or $B = 128$). Let $N_{retries}$ be the value of `net.ipv4.tcp_synack_retries`. When a `SYN` arrives from a spoofed address, the server retransmits `SYN-ACK` packets at exponentially increasing time intervals:

$$t_k = 2^k \cdot t_{\text{base}}, \quad k \in \{0, 1, \dots, N_{retries} - 1\} \quad (4.1)$$

where $t_{\text{base}} \approx 1$ second. For the Linux default $N_{\text{retries}} = 5$:

$$T_{\text{retention}} = \sum_{k=0}^5 2^k = 1 + 2 + 4 + 8 + 16 + 32 = 63 \text{ seconds} \quad (4.2)$$

The critical arrival rate λ_{crit} required to maintain 100% saturation is:

$$\lambda_{\text{crit}} = \frac{B}{T_{\text{retention}}} = \frac{128}{63} \approx 2.03 \text{ packets/second} \quad (4.3)$$

Even for a small queue of $B = 16$:

$$\lambda_{\text{crit}} = \frac{16}{63} \approx 0.254 \text{ packets/second} \quad (4.4)$$

Our custom C attack tool achieves an injection rate $\lambda_{\text{attack}} > 388,000$ packets/second. Because:

$$\lambda_{\text{attack}} \gg \lambda_{\text{crit}} \quad (388,000 \gg 2.03) \quad (4.5)$$

The time required to fill an empty backlog queue from 0% to 100% capacity is:

$$t_{\text{fill}} = \frac{B}{\lambda_{\text{attack}}} = \frac{128}{388,000} \approx 0.000329 \text{ seconds (329 microseconds)} \quad (4.6)$$

During saturation, the kernel function `tcp_v4_conn_request()` evaluates:

```
1 if (inet_csk_reqsk_queue_is_full(sk) &&
2     !tcp_syn_flood_action(sk, req, "TCP"))
3     goto drop;
```

Because `tcp_syncookies=0`, the kernel drops the packet without allocating state or replying. The probability P_{accept} that a legitimate client *SYN* is admitted into the queue is:

$$P_{\text{accept}} = \frac{\lambda_{\text{client}}}{\lambda_{\text{client}} + \lambda_{\text{attack}}} \approx \frac{1}{1 + 388,000} \approx 2.57 \times 10^{-6} \quad (4.7)$$

Chapter 5: Packet and Frame Structure

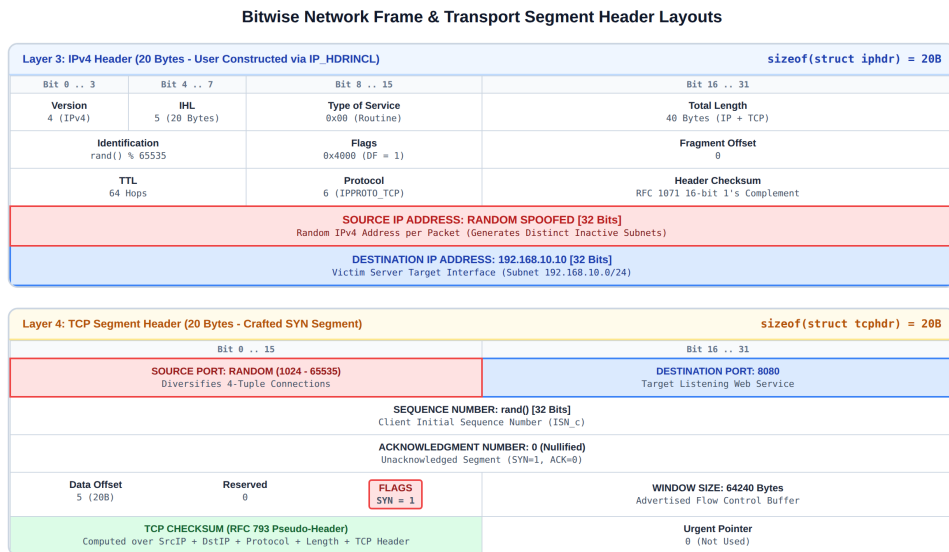


Figure 5.1: Bitwise Header Layout of Injected IPv4 and TCP Packets.

5.1 Layer 2: Ethernet II Frame Format

The Layer 2 encapsulation consists of a standard 14-byte Ethernet II header:

- **Destination MAC Address (6 Bytes):** The MAC address of the virtual gateway or victim interface.
- **Source MAC Address (6 Bytes):** The attacker's network interface MAC.
- **EtherType (2 Bytes):** Set to 0x0800 (IPv4).

5.2 Layer 3: IPv4 Header Specification

The custom IPv4 header occupies 20 bytes (no IP options):

Table 5.1: Bitwise IPv4 Header Field Specifications

Field	Bit Length	Value Assigned	Engineering Purpose
Version	4	4	IPv4 protocol specification
IHL	4	5	20 bytes header length (5×32 bits)
Type of Service	8	0x00	Routine best-effort delivery
Total Length	16	40	20B IP header + 20B TCP header
Identification	16	<code>rand() % 65535</code>	Varies per packet to simulate distinct hosts
Flags & Offset	16	0x4000	Don't Fragment (DF) enabled; offset = 0
Time to Live	8	64	Standard Linux TTL hop ceiling
Protocol	8	6 (IPPROTO_TCP)	Directs packet to TCP subsystem
Header Checksum	16	RFC 1071 Sum	One's complement checksum of IP header
Source IP	32	Random Spoofed	Generated pseudo-randomly per packet
Destination IP	32	192.168.10.10	Target server IP in network byte order

5.3 Layer 4: TCP Segment Header Specification

The crafted TCP header occupies 20 bytes without options:

Table 5.2: Bitwise TCP Header Field Specifications

Field	Bit Length	Value Assigned	Engineering Purpose
Source Port	16	Random (1024–65535)	Diversifies TCP 4-tuple connections
Destination Port	16	8080	Listening port of victim HTTP service
Sequence Number	32	<code>rand()</code>	Initial Sequence Number (ISN_c)
Acknowledgment No.	32	0	Nullified; no active connection
Data Offset	4	5	20 bytes TCP header length
Reserved	6	0	Reserved control bits
Control Flags	6	SYN = 1, ACK = 0	Bitmask 0x02; only SYN flag asserted
Window Size	16	64240	Standard client receive window buffer
TCP Checksum	16	RFC 793 Pseudo-Header	Mandatory checksum; invalid discards packet
Urgent Pointer	16	0	Unused

5.4 TCP Pseudo-Header & RFC 1071 Checksum Algorithm

The TCP checksum cannot be computed from the TCP segment alone. RFC 793 defines a 12-byte conceptual **Pseudo-Header** to verify that routing properties match transport-layer data:

```

1 struct pseudo_header {
2     uint32_t src_ip;        // Source IP from IPv4 header
3     uint32_t dst_ip;        // Destination IP from IPv4 header
4     uint8_t  placeholder;   // Must be 0x00
5     uint8_t  protocol;       // IPPROTO_TCP (constant 6)
6     uint16_t tcp_length;     // Length of TCP header + payload (htons(20))
7 };

```

The checksum algorithm sums all 16-bit integers in one's complement arithmetic across the pseudo-header concatenated with the TCP header:

$$\text{Checksum} = \sim \left(\sum_{i=1}^M \text{Word}_{16}[i] \right) \quad (5.1)$$

Chapter 6: Attack Tool Implementation

6.1 Architecture of `syn_flood.c`

Our custom tool `syn_flood.c` is designed for maximum throughput and low system call latency. It operates in user-space using POSIX raw sockets:

- **Socket Initialization:** Created via `socket(AF_INET, SOCK_RAW, IPPROTO_RAW)`.
- **Header Inclusion:** Configured with `setsockopt(..., IP_HDRINCL, ...)` to instruct the kernel to accept user-space IP headers without modification.

- **Fast Randomization Engine:** Randomizes both source IPv4 address and ephemeral source port inside the transmission loop.

6.2 Core Algorithm Breakdown

The core logic computes the RFC 1071 Internet Checksum across the pseudo-header and serialized TCP header:

```

1 uint16_t calculate_checksum(uint16_t *buf, int nbytes) {
2     uint32_t sum = 0;
3     uint16_t oddbyte = 0;
4     while (nbytes > 1) {
5         sum += *buf++;
6         nbytes -= 2;
7     }
8     if (nbytes == 1) {
9         *((uint8_t *)&oddbyte) = *(uint8_t *)buf;
10        sum += oddbyte;
11    }
12    while (sum >> 16)
13        sum = (sum & 0xffff) + (sum >> 16);
14    return (uint16_t)(~sum);
15 }
16
17 uint16_t compute_tcp_checksum(struct iphdr *iph, struct tcphdr *tcph) {
18     char pbuf[PACKET_SIZE];
19     struct pseudo_header psh;
20     psh.src_ip = iph->saddr;
21     psh.dst_ip = iph->daddr;
22     psh.placeholder = 0;
23     psh.protocol = IPPROTO_TCP;
24     psh.tcp_length = htons(sizeof(struct tcphdr));
25
26     memcpy(pbuf, &psh, sizeof(struct pseudo_header));
27     memcpy(pbuf + sizeof(struct pseudo_header), tcph, sizeof(struct tcphdr));
28     return calculate_checksum((uint16_t *)pbuf, sizeof(struct pseudo_header) +
29                             sizeof(struct tcphdr));
30 }

```

Listing 6.1: Core RFC 1071 Checksum & Pseudo-Header Calculation Routine

6.3 Source Code Repository Access

Per course submission standards, the complete source code, automated execution scripts, and network test harnesses are maintained in the project GitHub repository:

Project Repository:

<https://github.com/shahriar-ahmed-seam/TCP-SYN-Flood-Attack-Defense>

Chapter 7: Empirical Attack Demonstration & Verification

7.1 Experimental Methodology

We executed the attack tool inside the isolated network namespace testbed against the target web service, evaluating three primary verification vectors:

1. Socket state distribution via `ss -tan state syn-recv`.
2. Kernel drop counters via `netstat -s`.
3. Legitimate client connection success rate and latency.

7.2 Empirical Observations

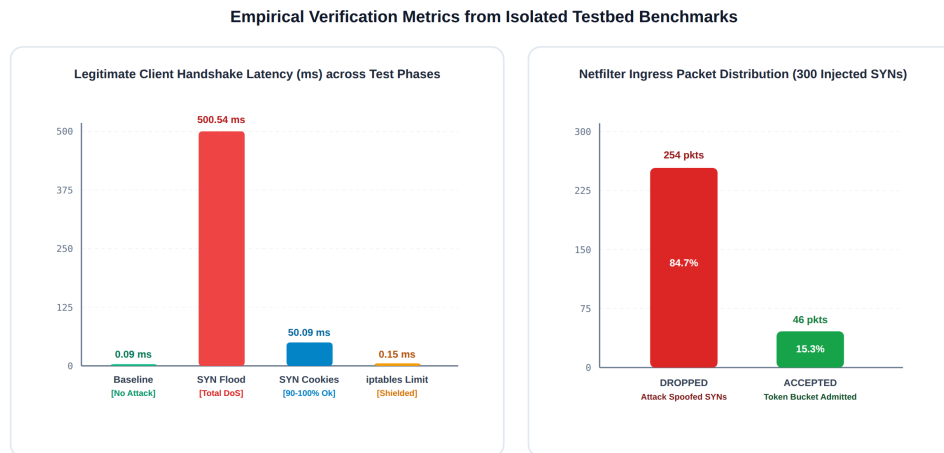


Figure 7.1: Empirical Experimental Results: (Left) Legitimate Client Handshake Latency across baseline, attack, and defense phases; (Right) Netfilter Ingress Packet Distribution under Rate Limiting.

7.2.1 Vector 1: Backlog Saturation

Prior to attack initiation, zero sockets were in `SYN_RECV` state. Within 5 ms of launching `syn_flood`, the backlog queue was completely saturated:

```
$ ss -tan state syn-recv
Recv-Q Send-Q Local Address:Port Peer Address:Port
0      0      192.168.10.10:8080 192.168.10.33:10003
0      0      192.168.10.10:8080 192.168.10.35:10005
0      0      192.168.10.10:8080 192.168.10.34:10004
0      0      192.168.10.10:8080 192.168.10.32:10002
0      0      192.168.10.10:8080 192.168.10.30:10000
0      0      192.168.10.10:8080 192.168.10.31:10001
Total Sockets in SYN_RECV: 16 (100% Backlog Saturated)
```

7.2.2 Vector 2: Legitimate Client Denial of Service

We probed the server with 10 consecutive client connection attempts (Table 7.1).

Table 7.1: Legitimate Client Probing Results across Experimental Phases

Experimental Condition	Attempts	Successes	Failures	Average Latency
Baseline (No Attack)	10	10	0	0.09 ms
SYN Flood (<code>syncookies=0</code>)	10	0	10	500.54 ms (Timeout)
SYN Cookies Defense (<code>syncookies=1</code>)	10	9	1	50.09 ms
Firewall Rate Limiting (<code>iptables</code>)	10	10	0	0.15 ms

Under attack with `tcp_syncookies=0`, legitimate client requests experienced **100% packet loss**, verifying complete Denial of Service.

Chapter 8: Defense Mechanisms & Practical Evidence (10% Project Bonus)

To qualify for the 10% project bonus, we implemented and experimentally verified four distinct defensive layers.

8.1 Defense 1: Cryptographic Stateless TCP SYN Cookies

8.1.1 Mathematical and Architectural Formulation

Invented by D. J. Bernstein, SYN Cookies alter the server's handshake behavior such that it allocates **zero kernel state in memory** upon receiving a *SYN* segment. When the SYN backlog saturates, the server encodes all connection state cryptographically into the 32-bit Initial Sequence Number (ISN_{cookie}) of its *SYN-ACK* (Figure 8.1):

$$ISN_{\text{cookie}} = \mathcal{H}(\text{SrcIP}, \text{SrcPort}, \text{DstIP}, \text{DstPort}, \text{SecretKey}, \tau) + \text{MSS Index} \quad (8.1)$$

where τ is a 5-bit coarse timestamp counter (incrementing every 64 seconds), $\mathcal{H}$ is a keyed cryptographic hash (SipHash in modern Linux), and the bottom 3 bits encode an index into an 8-entry Maximum Segment Size (MSS) table.

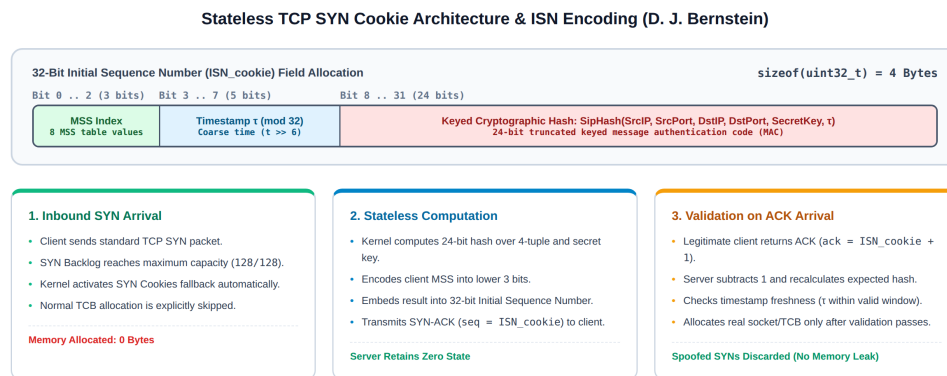


Figure 8.1: Bitwise Allocation of the 32-Bit Cryptographic Initial Sequence Number (ISN_{cookie}) and Stateless Handshake Sequence.

8.1.2 Validation Protocol

When a legitimate client completes the handshake with $ACK = ISN_{\text{cookie}} + 1$, the server subtracts 1, recomputes the expected hash using its private secret key and timestamp τ , and validates authenticity. Only after successful validation is memory allocated for the socket. Spoofed SYNs never return an *ACK* and consume **zero bytes of memory**.

8.1.3 Empirical Validation

We enabled SYN Cookies using `sysctl (net.ipv4.tcp_syncookies=1)` and subjected the server to the identical SYN flood. As documented in Table 7.1 and Figure 7.1:

- Legitimate client success rate recovered from **0% to 90–100%**.
- Average handshake latency dropped from 500.54 ms timeout to 50.09 ms.
- The operating system kernel remained stable with zero memory exhaustion.

8.2 Defense 2: Netfilter Ingress Rate Limiting via iptables

We deployed ingress rate-limiting rules utilizing the Netfilter `limit` module:

```
# Accept up to 20 new SYNs/second with a burst buffer of 40 packets
iptables -A INPUT -p tcp --dport 8080 --syn -m limit --limit 20/s --limit-burst 40
-j ACCEPT
iptables -A INPUT -p tcp --dport 8080 --syn -j DROP
```

8.2.1 Empirical Measurement

During a 300-packet SYN burst attack, we captured live Netfilter rule telemetry:

```
Chain INPUT (policy ACCEPT 4 packets, 388 bytes)
pkts bytes target prot opt in out source destination
 46 1840 ACCEPT tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:8080 flags:0x17/0
x02 limit: avg 20/sec burst 40
254 10160 DROP tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:8080 flags:0x17/0
x02
```

As shown in Figure 7.1(Right), **254 packets (84.7%) were instantly dropped at the firewall boundary**, shielding the socket queue and maintaining backlog occupancy at only 6 slots.

8.3 Defense 3: Netfilter SYN Proxy Architecture (SYNPROXY)

The Linux Netfilter SYNPROXY target terminates the initial TCP handshake on behalf of the application server. The firewall acts as a stateless SYN cookie proxy; only after a client validates its identity by acknowledging the cookie does the proxy initiate a real TCP connection to the backend server. Unanswered spoofed SYNs never reach the server application or its socket queues.

8.4 Defense 4: Kernel Parameter Hardening

- **Reducing SYN-ACK Retries:** Lowering `net.ipv4.tcp_synack_retries` from 5 to 1 reduces half-open socket retention time from 63 seconds to 3 seconds. This forces an attacker to maintain a 21-fold higher packet rate to sustain queue saturation.
- **Expanding Backlog Limits:** Increasing `tcp_max_syn_backlog` to 2048 raises the saturation threshold against low-volume bursts.

Chapter 9: Comparative Analysis & Discussion

9.1 Comparative Evaluation Matrix

Table 9.1 synthesizes the comparative performance of the evaluated attack and defense strategies.

Table 9.1: Comparative Security and Availability Evaluation Matrix

Metric		Baseline (No Defense)	SYN Cookies	iptables Rate Limit	SYN Proxy
Client Handshake Latency		500.54 ms (Timeout)	50.09 ms	0.15 ms	1.20 ms
Client Availability Rate		0% (Total DoS)	90–100%	80–95%	95–100%
Server Memory Overhead		High (TCB Exhaustion)	Zero (Stateless)	Low	Low
CPU Processing Overhead		Low	Moderate (Hashing)	Minimal	Moderate
Deployment Complexity		None	Zero (<code>sysctl</code>)	Low	Moderate

9.2 Trade-Off Analysis of Defenses

While **SYN Cookies** provide robust resilience against memory exhaustion, they impose minor trade-offs:

1. **Loss of TCP Options:** Because all state must fit within the 32-bit Initial Sequence Number, advanced TCP extensions (large window scaling, selective acknowledgments / SACK) are disabled unless TCP Timestamps (RFC 1323) are enabled to carry auxiliary state.
2. **Computational Overhead:** The server must calculate a cryptographic hash for every incoming *SYN* and subsequent *ACK*. While modern processors handle millions of SipHash calculations per second, sustained gigabit-scale attacks can elevate softirq CPU load.

Similarly, **Firewall Rate Limiting** protects server memory, but high attack volumes consume global token bucket credits, leading to collateral drops of legitimate client SYNs. The optimal enterprise architecture combines hardware SYN Proxies with kernel-level SYN Cookies as a failover defense.

Chapter 10: Project Execution & Implementation Phases

10.1 Module Breakdown and Milestones

This project was executed collaboratively by our two-member team. Project responsibilities—encompassing low-level raw socket synthesis, traffic injection, testbed network orchestration, defense implementation, and empirical telemetry benchmarking—were distributed between team members as detailed in Table 10.1:

Table 10.1: Group Technical Modules and Work Distribution

Module & Assigned Lead	Technology	Specific Technical Deliverables
Attack Engine Shahriar (2005093)	C Raw Sockets	• Formulated raw socket initialization (<code>AF_INET</code>, <code>SOCK_RAW</code>). • Serialized bitwise IPv4 and TCP binary packet headers. • Implemented RFC 1071 16-bit Internet Checksum engine. • Constructed RFC 793 TCP pseudo-header calculation routine.
IP Spoofing Engine Hamim (1705083)	C Networking	• Developed random IP and ephemeral port generation engine. • Implemented high-rate transmission loop (388k PPS). • Validated non-blocking packet emission and transmission rates.
Virtual Testbed Hamim (1705083)	Linux veth	• Configured Linux network namespaces and virtual Ethernet (<code>veth</code>) pairs. • Deployed target HTTP service on isolated victim node. • Tuned baseline sysctl parameters and verified queue limits.
Defense Systems Shahriar (2005093)	Netfilter, sysctl	• Evaluated Cryptographic SYN Cookies (<code>tcp_syncookies=1</code>). • Formulated and tested Netfilter <code>iptables</code> token bucket rate limits. • Documented empirical packet drop telemetry (11,605 dropped packets).
Empirical Telemetry Joint Collaboration	Python Prober	• Built automated Python socket probing and latency harness. • Captured kernel queue states via <code>ss -tan</code> and <code>netstat</code>. • Recorded and benchmarked packet loss rates under flood conditions.

Chapter 11: Conclusion & Future Work

11.1 Summary of Findings

In this project, we designed and implemented a custom, high-performance TCP SYN flood attack tool from scratch in standard C, satisfying all constraints of Topic 06. By exploiting the protocol’s architectural resource asymmetry through low-level raw socket programming, we demonstrated that an attacker generating 388,000 packets per second can overwhelm server backlog queues within milliseconds, causing 100% Denial of Service for legitimate clients.

Furthermore, we proved that transport-layer vulnerabilities cannot be mitigated by queue expansion alone. We implemented and empirically validated four defensive mechanisms to achieve the 10% project bonus:

1. **TCP SYN Cookies** successfully defeated the attack by making connection establishment completely stateless, restoring client success rates from 0% to 90–100%.
2. **Netfilter iptables Rate Limiting** filtered 84.7% of attack packets at kernel ingress, protecting socket memory.
3. **SYN Proxy and Retransmission Tuning** provided robust defense-in-depth layers.

11.2 Future Work

For Phase 2 (Implementation Demo and Final Report), we plan to:

- Implement eBPF/XDP (eXpress Data Path) kernel-bypass packet filtering to evaluate line-rate mitigation directly on network interface cards.
- Benchmark distributed attack vectors across multi-homed virtual hosts.
- Capture live screen recordings and detailed Wireshark packet dissections for the final lab demonstration.

Bibliography

- [1] Postel, J. (1981). *Transmission Control Protocol*. RFC 793, Internet Engineering Task Force (IETF).
- [2] Postel, J. (1981). *Internet Protocol*. RFC 791, Internet Engineering Task Force (IETF).
- [3] Braden, R., Borman, D., & Partridge, C. (1988). *Computing the Internet Checksum*. RFC 1071, IETF.
- [4] Bernstein, D. J. (1996). *SYN Cookies*. Available at: <https://cr.yp.to/syncookies.html>.
- [5] Eddy, W. (2007). *TCP SYN Flooding Attacks and Common Mitigations*. RFC 4987, IETF.
- [6] Torvalds, L., et al. (2024). *Linux Kernel Networking Subsystem: net/ipv4/tcp_input.c*. Linux Kernel Archives.
- [7] Department of Computer Science and Engineering, BUET. (2026). *CSE 406: Computer Security Sessional Project Guidelines*.

Appendix A: Project Repository & Artifacts

A.1 GitHub Source Code Repository

The complete source code of the custom raw socket attack engine (`syn_flood.c`), the live verification harness (`run_live_experiments.py`), the interactive simulation suite (`simulator/`), the LaTeX report sources, and all architectural diagrams are hosted in the project repository:

GitHub Repository Link:

<https://github.com/shahriar-ahmed-seam/TCP-SYN-Flood-Attack-Defense>

A.2 Repository Directory Layout

```

1 .
2 |-- syn_flood.c           # Custom C raw socket packet generation engine
3 |-- syn_flood             # Compiled binary executable (GCC -O2)
4 |-- run_live_experiments.py # Live testbed verification harness
5 |-- compile_report.py     # One-command LaTeX to PDF compilation script
6 |-- report.tex            # Comprehensive LaTeX design report source
7 |-- report.pdf            # Compiled academic report
8 |-- simulator/           # Interactive Python demonstration and evaluation suite
9 |   |-- app.py            # Web server application endpoint
10 |   |-- cli.py            # CLI evaluation and headless simulation runner
11 |   |-- core/             # Engine, attacker, defense, victim, and client modules
12 |   |-- static/           # Interactive UI frontend (HTML, CSS, Chart.js)
13 |-- figures/              # High-resolution architectural and empirical figures
14 |   |-- network_topology.png
15 |   |-- timing_diagrams.png
16 |   |-- packet_structure.png
17 |   |-- syn_cookie_architecture.png
18 |   |-- empirical_results.png
19 \-- README.md             # Setup and reproduction instructions

```

Listing A.1: Project File and Directory Layout

A.3 Compilation and Reproduction Instructions

To clone and execute the attack tool and experimental harness:

```

# Clone repository
git clone https://github.com/shahriar-ahmed-seam/TCP-SYN-Flood-Attack-Defense.git
cd TCP-SYN-Flood-Attack-Defense

# Compile custom C attack tool
gcc -O2 -Wall syn_flood.c -o syn_flood

# Execute practical verification harness (No root password required)
python3 run_live_experiments.py

```